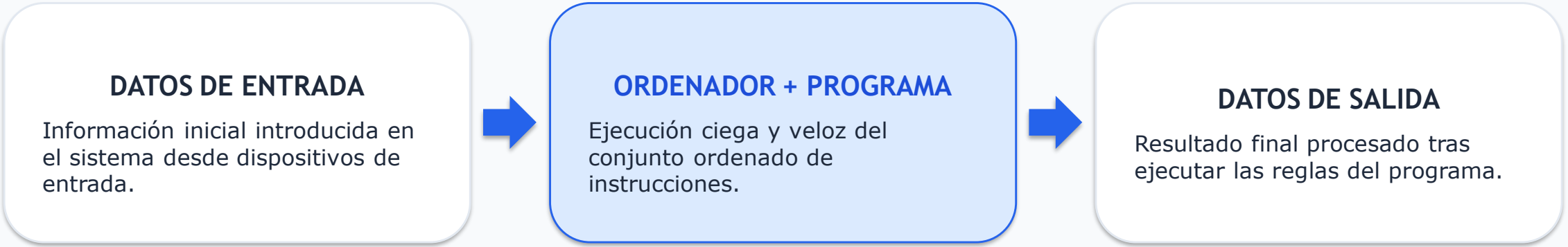
Herramientas y Fundamentos del Desarrollo de Software

Unidad Temática 1 (RA1) — El Programa Informático y Ecosistema de Desarrollo

Síntesis basada en las fuentes A-ED-UT1-RA1.pdf y A-ED-UT1-RA1-CEf.pdf

Un programa convierte un conjunto ordenado de instrucciones y datos en información útil



Principios Fundamentales de la Informática:

- Informática: Definida como el tratamiento automático de la información mediante ordenadores.
- Naturaleza del ordenador: Carece de inteligencia propia; obedece ciegamente instrucciones simples a altísima velocidad y con mínima probabilidad de error.
- Programa Informático: Conjunto ordenado de instrucciones estructuradas según las reglas y símbolos de un lenguaje de programación.

Variables, constantes y expresiones gestionan la memoria y la lógica en ejecución

1. Variables

Nombre / Identificador
Referencia simbólica a una zona de memoria.

Memoria Asociada
Posiciones contiguas en la memoria RAM principal.

Valor Modificable
El contenido puede cambiar durante la ejecución.

Tipo de Dato
Define el rango de valores permitidos (ej. entero, alfabético).

2. Constantes

Inmutabilidad
Guarda un valor fijo que NO puede cambiarse durante la ejecución.

Estructura de Memoria
Misma asociación a memoria y tipo que una variable.

Propósito
Aporta seguridad y claridad en el código (ej. PI = 3.1416).

Asignación
Se define al inicio con su tipo y valor inalterable.

3. Expresiones y Operadores

Composición
Combinación de constantes, variables, operadores y paréntesis.

Evaluación de Operadores
Evaluados según orden estricto de prioridad.

Tipos de Operadores
Aritméticos, alfanuméricos, relacionales y lógicos.

Resultado y Tipo
Produce un valor final de un tipo concreto (ej. 5 + 4 * 2 → 13).

Las instrucciones de definición, E/S y control determinan el flujo del programa

Instrucciones de Definición (Declaración de Datos)

Instrucciones de Entrada / Salida (E/S)

Instrucciones de Control de Flujo

El hardware ejecuta la lógica mientras el software de sistema y aplicación coordinan el equipo

HARDWARE (Parte Tangible)

- Microprocesador (CPU): 'El cerebro' que realiza los cálculos.
- Placa Base: Soporte interno donde se conectan los componentes.
- Memoria RAM: Almacenamiento volátil para datos en ejecución.
- Disco Duro / Unidades Ópticas: Almacenamiento permanente.
- Fuente de Alimentación: Conversión de tensión alterna a continua.
- Periféricos (E/S): Monitor, teclado, ratón y tarjetas de expansión.

SOFTWARE (Elementos Lógicos)

1. Software de Sistema:
- Sistema Operativo (Windows, Linux, macOS).
- Controladores de dispositivos (Drivers).
- Utilidades del sistema para el control del hardware.

2. Software de Aplicación:
- Programas para tareas concretas del usuario.
- Herramientas ofimáticas, diseño gráfico, contabilidad y ocio.

Cinco capas organizan la arquitectura desde el hardware físico hasta las aplicaciones

Capa 5: Programas de Aplicación
Editores de texto, navegadores y datos de uso habitual del usuario final.

Capa 4: Utilidades de Desarrollo
Entornos y herramientas empleadas por programadores para crear aplicaciones.

Capa 3: Sistema Operativo
Windows, Unix/Linux, macOS. Gestiona el hardware y ejecuta las capas superiores.

Capa 2: Firmware / BIOS
Chip en placa base. Intermediario HW/SW; comprueba hardware y carga el SO en RAM.

Capa 1: Hardware
Base física (circuitos integrados, CPU, RAM) que ejecuta las instrucciones.

La virtualización aísla sistemas invitados sobre un anfitrión optimizando recursos

Estructura de Virtualización

- Máquina Virtual: Simulación de un sistema informático independiente dentro del equipo real.
- S.O. Anfitrión: Sistema operativo real instalado sobre el hardware físico.
- Software de Virtualización: Aplicación sobre el SO Anfitrión (ej. VirtualBox, VMWare).
- S.O. Invitado: Sistema operativo que funciona dentro de cada máquina virtual con recursos simulados.

Balance Operativo

Ventajas Clave:

+ Pruebas seguras sin riesgo de degradar el SO anfitrión.
+ Simulación de redes completas sin infraestructura física.
+ Alta portabilidad (copia directa de ficheros de la VM).
+ Compatibilidad con programas de otros entornos.

Inconvenientes:

- Demanda de hardware potente (apropiación de recursos reales, especialmente memoria RAM).

La arquitectura Von Neumann coordina memoria, buses, ALU y Unidad de Control

1. Memoria Principal (RAM) y Buses

2. Unidad Aritmético-Lógica (UAL / ALU)

3. Unidad de Control (UC) — Coordinación de la Ejecución

Los lenguajes de programación evolucionan del código máquina a la sintaxis declarativa

1. Lenguaje Máquina

Cercano al HW

Código binario y numérico nativo de la CPU.
Totalmente dependiente de la arquitectura interna.

2. Lenguaje Ensamblador

Simbólico / Bajo Nivel

Símbolos mnemónicos con relación 1 a 1 respecto al lenguaje máquina.
Dependiente de CPU.

3. Lenguaje de Alto Nivel

Portable / Legible

Independiente de la máquina (C, Pascal, Java).
Fácil lectura humana; traducido a decenas de instrucciones máquina.

4. Lenguajes Declarativos

Cercano al Humano

Expresan 'qué hacer' en lugar de 'cómo hacerlo'.
Basados en órdenes (SQL, SAS, SPSS).

Compiladores e intérpretes traducen el código mediante enfoques previos o en tiempo real

Compiladores (Ej. C, Pascal)

Intérpretes (Ej. LISP, PROLOG)

Modelo Híbrido: El Lenguaje Java

El ecosistema moderno integra IDEs, Git, metodologías ágiles y herramientas especializadas

Funcionalidad en el Desarrollo de Software	Herramientas de Ejemplo (Anexo CEf)
Automatización de pruebas del software	JUnit
Construcción automática del software Java	Maven, Gradle
Control de versiones del software	Git
Generación automática de documentación desde comentarios Java	Javadoc
Entorno de desarrollo integrado (IDE) / Depuración / Ágil / UML	Eclipse, Netbeans | Depurador | Scrum, Kanban, XP | StarUML